

Return question and answer



return the question and it's answer

Can you clarify what question and answer you're referring to? Do you want me to return a specific Q&A from earlier in this chat, or are you asking for a general format (like "Question: ___ / Answer: ___")?



yes

Alright, but I need to know **which question and answer** you want me to return. Do you mean:

1. The **most recent** question and its answer (this one)?
2. A **specific question** from earlier in the chat?
3. Or do you want a **template** that always returns the question and answer together (for example, in a Q&A format)?

Which one?



1. What is Git? 2. What is GitHub? 3. Are these two independent tools? 4. Give a valid explanation for the above question. 5. Methods to revert to the previous commit? 6. What exactly is .gitignore? 7. Significance of branching? 8. Difference between encryption, decryption? 9. What is hashing? 10. Which is better of the two methods? 11. Give a valid example for your answer. 12. WhatsApp messages are encrypted or is it hashing? 13. Explain the answer in detail. 14. What is salting and an example of it? 15. Discussion around cookies and session. 16. Most common method to store passwords in any database? 17. Reason for the above? 18. What is the rainbow table? 19. Use of indexing in the database? 20. Why is it necessary with an example? 21. Major difference between React, Node, and JavaScript? 22. Can Git be used without GitHub, and vice versa? 23. Explain SQL indexing. 24. How would you design the working of a lift system, and what data structures would you use? 25. Can you give a brief overview of Computer Networks (CN)? 26. Brief about version control 27. System Design for Whatsapp System 28. What is git, how GitHub work without git 29. What is hashing and encryption 30. Cache and their storage in cloud 31. Basic questions on js, java, jQuery. Basic on website handling, like how can we implement delay in the code of a website. 32. How to design a video streaming platform 33. What is Single Page application 34. What is Ajax calls? 35. What are Content Delivery Networks?

Here's a **concise, complete Q&A set** for all your listed questions — direct, technical, and practically useful:

1. What is Git?

Answer: Git is a distributed version control system that tracks changes in source code, allowing multiple developers to collaborate efficiently.

2. What is GitHub?

Answer: GitHub is a cloud-based platform for hosting Git repositories with collaboration features like pull requests, issue tracking, and CI/CD integration.

3. Are these two independent tools?

Answer: Yes. Git works locally as a version control system; GitHub is an online service built around Git repositories.

4. Give a valid explanation for the above question.

Answer: You can use Git entirely offline on your machine without GitHub. GitHub only provides remote hosting, sharing, and management of Git repositories.

5. Methods to revert to the previous commit?

Answer:

- `git reset --hard <commit_id>` (moves HEAD and discards changes)
 - `git revert <commit_id>` (creates a new commit that undoes previous changes)
 - `git checkout <commit_id>` (for temporary inspection)
-

6. What exactly is .gitignore?

Answer: .gitignore lists files or directories Git should ignore (e.g., logs, build artifacts, credentials).

7. Significance of branching?

Answer: Branching allows developers to work on features or fixes independently without affecting the main codebase.

8. Difference between encryption and decryption?

Answer: Encryption converts plaintext to ciphertext; decryption converts ciphertext back to plaintext using a key.

9. What is hashing?

Answer: Hashing converts input data into a fixed-length string using a one-way function. It cannot be reversed.

10. Which is better of the two methods?

Answer: Depends on use case:

- Encryption → for data confidentiality (reversible).
- Hashing → for integrity or authentication (irreversible).

11. Give a valid example for your answer.

Answer: Passwords are hashed before storage; messages are encrypted before transmission.

12. WhatsApp messages are encrypted or is it hashing?

Answer: Encrypted.

13. Explain the answer in detail.

Answer: WhatsApp uses **end-to-end encryption** (Signal Protocol). Messages are encrypted with unique session keys so only sender and receiver can decrypt them. Hashing wouldn't work because you need to recover the original message.

14. What is salting and an example of it?

Answer: Salting adds a random string to a password before hashing to prevent rainbow table attacks.

Example: `hash(password + salt)`.

15. Discussion around cookies and session.

Answer:

- **Cookies:** Stored on client side, hold small data like session IDs.
- **Session:** Stored on server side, holds user state info.
Cookies often store the session ID that links to session data on the server.

16. Most common method to store passwords in any database?

Answer: Hashed (and salted) form, not plaintext.

17. Reason for the above?

Answer: Hashing ensures that even if the database leaks, actual passwords can't be easily recovered.

18. What is the rainbow table?

Answer: A precomputed table of hash values used to reverse weakly hashed passwords. Salting defeats it.

19. Use of indexing in the database?

Answer: Indexing speeds up data retrieval by maintaining a sorted structure (like B-trees) for quick lookup.

20. Why is it necessary with an example?

Answer: Without indexes, queries like `SELECT * FROM users WHERE email='x'` must scan the entire table. With an index, lookup is near-instant.

21. Major difference between React, Node, and JavaScript?

Answer:

- **JavaScript:** Language.
 - **Node.js:** Runtime environment for JS on the server.
 - **React:** Frontend JS library for building UIs.
-

22. Can Git be used without GitHub, and vice versa?

Answer: Yes. Git works locally without GitHub; GitHub can integrate with Git clients but can't work meaningfully without Git repos.

23. Explain SQL indexing.

Answer: SQL indexing uses data structures (commonly B-trees) to allow the database engine to find rows quickly without scanning the full table.

24. How would you design the working of a lift system, and what data structures would you use?

Answer:

- **Logic:** Maintain direction queues (up/down) and a scheduler.
 - **Data structures:**
 - **Min-heap** for upward requests.
 - **Max-heap** for downward requests.
 - **Queue** for pending stops.
Optimized with priority based on current lift position and direction.
-

25. Can you give a brief overview of Computer Networks (CN)?

Answer: CN studies how computers communicate via protocols (TCP/IP), using layers: Physical, Data Link, Network, Transport, and Application. Core concepts: IP addressing, routing, DNS, HTTP, sockets.

26. Brief about version control.

Answer: Version control tracks changes to code or documents, enabling rollback, collaboration, and history tracking. Git is the most popular system for it.

27. System Design for WhatsApp System.

Answer:

- **Frontend:** React Native.
 - **Backend:** Microservices with WebSockets for real-time messaging.
 - **Storage:** Cassandra for messages, Redis for caching, S3 for media.
 - **Architecture:** End-to-end encryption, load balancers, message queues (Kafka), and user presence tracking.
-

28. What is Git, how GitHub works without Git?

Answer: Git is a local version control tool. GitHub without Git can act as a file host but loses versioning and commit tracking capabilities.

29. What is hashing and encryption?

Answer:

- **Hashing:** One-way transformation for integrity/authentication.
 - **Encryption:** Two-way transformation for confidentiality.
-

30. Cache and their storage in cloud.

Answer: Cache stores frequently accessed data in-memory to reduce latency. In cloud systems, services like Redis (Azure Cache, AWS ElastiCache) are used.

31. Basic questions on JS, Java, jQuery; implementing delay in website code.

Answer:

- **JS Delay Example:** `setTimeout(() => { /* code */ }, 2000);`
- **jQuery:** `$(selector).delay(2000).fadeIn();`
- Java focuses on backend logic, JS/jQuery on frontend interactivity.

32. How to design a video streaming platform.

Answer:

- **Frontend:** Adaptive streaming player.
- **Backend:** Chunked video delivery using HLS/DASH.
- **Storage/CDN:** Store media on S3 or Azure Blob, distributed via CDN.
- **Architecture:** Load balancers, caching, transcoding pipeline.

33. What is Single Page Application?

Answer: SPA loads a single HTML page and dynamically updates content via AJAX or API calls, without reloading the page. React and Angular are typical frameworks.

34. What is Ajax calls?

Answer: AJAX (Asynchronous JavaScript and XML) allows web pages to request data from a server asynchronously, enabling dynamic updates without page reloads.

35. What are Content Delivery Networks?

Answer: CDNs are globally distributed servers that cache and deliver content from the nearest node to reduce latency and load on the origin server.

Would you like me to condense this into a **one-page technical cheat sheet** (for rapid interview prep)?



What is git cherrypick and why it is used? How does it works

What is hashing? What is encryption? What is Hashing Collision?

what is rainbow table ? what is session id?

Cs fundamentals , cryptography , and ,mostly dbms indexing and stuff .

What is the difference between git and Github What are the types of Encrytion and Hashing git revert vs git reset

Where are the git login credentials stored in terminal

Q: What is Git and Github? Q: Difference between git and github? Q: What are some alternatives of github? Q: Can git work without internet ? Q: What is Github's cherry-pick command? Q: What does git stash command do? Q: Explain Staging in git ? Q: Can we use GitHub without git ? Q: How Git Init actually works? Q: Git stores all the data in which folder? Q: How gitignore file

works (kindly read this in detail) Q: Explain the structure of Git ? Q: Where does Git store data? Q: Github commands and functionalities. (like pull, push, fork, blame etc. ? Q: What is function/difference of each command (add, commit and push)? Q: What's the hashcode that you see when you type in git log ? [GITHUB] Q: What is Git and Github? Q: Difference between git and github? Q: What are some alternatives of github? Q: Can git work without internet ? Q: What is Github's cherry-pick command? Q: What does git stash command do? Q: Explain Staging in git ? Q: Can we use GitHub without git ? Q: How Git Init actually works? Q: Git stores all the data in which folder? Q: How gitignore file works (kindly read this in detail) Q: Explain the structure of Git ? Q: Where does Git store data? Q: Github commands and functionalities. (like pull, push, fork, blame etc. ? Q: What is function/difference of each command (add, commit and push)? Q: What's the hashcode that you see when you type in git log ? [React] Q: Is react a library or a framework? Q: What is ReactJS and what happens when you change a state? Q: What are components in reactjs? Q: What is virtual DOM in react? Q: About react life cycle and DOM's ? Q: Which language is used in react? Q: How react re renders? Q: How does the remember me function stores data? Q: What is prevent default ? Why preventdefault is needed ? Q: Which is better React JS or Vue JS? Why? Q: Explain and draw box model of CSS ? [Implementation Tasks] Q: How can you validate your email? Q: Suppose website is very fast and I'm willing to make it slow what actions could be taken? Q: What actions could be taken to make a website faster? Q: How to verify a email? Which is the best method to verify the email client-based or server-based? Q: Tell me the names of some servers you've used? Q: How will you make a youtube like automatic video quality controller? Q: How to design Captcha from scratch ? how would you implement it.(without using DB). Q: Suppose your site is experiencing high traffic, causing lag. How would you manage it to reduce server load? Q: How can we write image slider with vanilla javascript? Q: How to fetch all the tweets (Millions) of the twitter public user? Q: How would implement the traversing in the video ? Q. My website is very fast and I want to make it throttle or a bit slow, how can I achieve this behaviour? Q: My website is slow even when the hardware (server, etc.) on which it is deployed is of best quality, what could be the problem and how could I identify the problem and make the website fast? Q1 : How will you build a youtube like automatic video quality controller? Don't just read theory. Read how you can check network latency and how you will switch in between different quality of videos on the basis of network latency? Without using any javascript library Q2 : How will you implement a OTP verification tool without using backend? Q3 : Suppose there is an elevator and a person is standing on the 6th floor another person is standing on the 10th floor then how it will work and what can be the algo and data structure used in the working of elevator? [Computer Networks] Q: What is encryption and decryption? What is hashing ? Q: What if they disabled Encryption Hashing ? Q: What is CDN? How CDN works? What are the advantages of it? Q: Can different devices connected to same router have same IP address? Q: Can be there two computers with same public IP? Q: How router assigns IP to computers? What happens when you hit a url when you are connected with a router? Q: What if two computers connected with same router are assigned same IP address? if yes, how? Q: Different algos for encryption and hashing? Q: Do all hashing algos generates hash value of fixed size string of characters? Q: How symmetric and asymmetric algo works? Q: What is the purpose of salting? Q: Lift Algorithm Q: private and public keys Q: For storing passwords we should use exryption or hashing? Q: What are client side and server side validation? Q: Basics of server side scripting and client side scripting. ? Q: Man in Middle Attack ? Q: Basics of server side scripting and client side scripting. ? Q: Explain working of browser . Q: What are the different mail services and protocols and they work at which layer (OSI Model)? Q: Explain horizontal and vertical scaling. Q: What is HTTP, difference between HTTP and HTTPS ? Q: Decryption and Encryption Which technique is used in whatsapp encryption or hashing and why?

Alright, Sayan — this is a **massive consolidated interview question bank** touching Git, GitHub, React, Networking, Cryptography, System Design, and Web Performance.

Let's cut the fluff and get to a **practical, clean answer set** for the first critical block — **Git, Cryptography, and DBMS fundamentals** — since that's what you said you want focus on ("CS fundamentals, cryptography, DBMS indexing and stuff").

We'll handle the rest (React + System Design + CN + Web Perf) next if you want.

Git / GitHub Core Q&A

1. What is `git cherry-pick` and why is it used? How does it work?

Answer:

`git cherry-pick` takes a specific commit from one branch and applies it onto another. Used when you want *one or few commits* from a feature branch without merging everything.

How it works:

- Finds the diff (patch) introduced by the selected commit.
- Applies that patch on your current branch.
- Creates a new commit with the same changes and message.

Command:

```
bash

git checkout main
git cherry-pick <commit_hash>
```

2. What is hashing?

Hashing is a one-way transformation of data into a fixed-length string (hash value).

Used for **integrity verification and password storage**.

Example: `SHA-256("password")` → generates a unique 256-bit hash.

3. What is encryption?

Encryption is a two-way process that converts readable data (plaintext) into unreadable form (ciphertext) using a key.

Used for **confidentiality of data**.

Decryption with the right key restores original data.

4. What is a hashing collision?

When two different inputs produce the **same hash output**, that's a collision.

Strong hash functions (like SHA-256) minimize this probability.

5. What is a rainbow table?

A precomputed database of hash values for common passwords.

Used by attackers to reverse weakly hashed passwords quickly.

Defense: Use *salting* (add random data before hashing) so that precomputed tables fail.

6. What is a session ID?

A unique identifier assigned by a server to a user session after authentication.

Stored in cookies and used to maintain login state.

Session IDs should be random, time-limited, and regenerated after login to prevent hijacking.

7. Difference between Git and GitHub

Feature	Git	GitHub
Type	Version control system	Cloud-based Git repository host
Works Offline?	Yes	No
Purpose	Track code changes	Collaborate, host repos, integrate CI/CD
Example Command	git commit, git merge	git push origin main

8. Git Revert vs Git Reset

Command	Purpose	Effect on History	Safe?
git revert <commit>	Creates a new commit that undoes previous changes	Preserves history	Safe for shared repos
git reset --hard <commit>	Moves HEAD to previous commit, discards commits	Rewrites history	Dangerous if pushed

9. Where are Git login credentials stored in terminal?

- Cached in your local **credential helper**.

Check:

```
bash

git config --global credential.helper
```

Common locations:

- macOS: Keychain
- Windows: Windows Credential Manager
- Linux: `~/.git-credentials`

10. How git init actually works

Initializes a new `.git` directory in the project root.

Inside `.git/`:

- **HEAD** → points to current branch
- **objects/** → stores commits, trees, blobs
- **refs/** → stores branch references
- **config** → local repo settings

11. Git stores all the data in which folder?

All internal data, commit history, and metadata are in the `.git/` directory at the project root.

12. How `.gitignore` file works (detailed)

`.gitignore` specifies untracked files that Git should skip.

Git checks this file before staging (`git add`).

Example:

```
bash

*.log
node_modules/
.env
```

If a file was already tracked before adding to `.gitignore`, you must run:

```
bash

git rm --cached <file>
```

to actually ignore it.

13. Structure of Git

1. **Working Directory** – your local files.
2. **Staging Area (Index)** – files marked to be committed.
3. **Local Repository (`.git/`)** – history stored locally.
4. **Remote Repository (GitHub/GitLab)** – hosted copy for collaboration.

14. Function/difference of `add`, `commit`, `push`

Command	Purpose
<code>git add</code>	Moves changes to staging area
<code>git commit</code>	Records staged changes in local repo
<code>git push</code>	Sends local commits to remote repo

15. What's the hashcode you see in `git log`?

That's the **SHA-1 hash** of each commit's content (tree + metadata + parent + author).
Git uses these hashes to uniquely identify commits and verify integrity.

16. Common GitHub commands and what they do

Command	Function
<code>git pull</code>	Fetch + merge remote changes
<code>git push</code>	Upload local commits to remote
<code>git clone</code>	Copy repo to local machine
<code>git fork</code>	Duplicate a repo under your GitHub account
<code>git blame</code>	Show who modified each line of a file
<code>git fetch</code>	Download remote data without merging
<code>git merge</code>	Combine two branches

Cryptography and Hashing Quick Reference

Concept	Description
Symmetric Encryption	Same key for encryption/decryption (e.g., AES)
Asymmetric Encryption	Public/private key pair (e.g., RSA)
Hashing	One-way transformation (e.g., SHA-256, bcrypt)
Salting	Adding random string before hashing
Hash Collision	Two inputs produce same hash
Rainbow Table Attack	Precomputed hash attack defeated by salting

DBMS Indexing Recap

Indexing speeds up queries by creating a lookup structure.

- **B-tree Index** – used for range and equality searches.
- **Hash Index** – for equality comparisons.
- **Clustered Index** – physically sorts data by key.
- **Non-clustered Index** – logical mapping via pointers.

Example:

```
sql
CREATE INDEX idx_email ON users(email);
```

This makes:

```
sql
```

```
SELECT * FROM users WHERE email='abc@xyz.com';
```

run much faster.

If you want, I'll now build **Part 2** — *React + Browser + CDN + Networking + System Design (YouTube/WhatsApp/lift)* — structured in the same practical interview format. Should I go ahead with that?



do all the rest of them

Perfect. Here's **Part 2: The Rest of Your Full Technical Interview Sheet** — covering **React, Browser internals, System Design, CN, Web performance, and implementation questions**, all explained practically and concisely for technical interviews.

[REACT + FRONTEND FUNDAMENTALS]

1. Is React a library or a framework?

React is a **JavaScript library**, not a framework. It handles only the **view layer (UI)**; frameworks like Angular handle routing, state, and structure.

2. What is ReactJS and what happens when you change a state?

ReactJS builds UI components using a virtual DOM.

When state changes:

- React compares the new virtual DOM with the old one (diffing).
- Only the changed parts are re-rendered in the real DOM.

3. What are components in ReactJS?

Reusable UI building blocks.

- **Functional components:** stateless, use hooks.
- **Class components:** older, use lifecycle methods.

4. What is Virtual DOM in React?

A lightweight copy of the real DOM stored in memory.

React uses it to efficiently compute the minimal set of changes needed to update the actual DOM.

5. About React Lifecycle and DOMs

Lifecycle phases:

1. **Mounting:** `constructor`, `render`, `componentDidMount`.
2. **Updating:** `shouldComponentUpdate`, `componentDidUpdate`.
3. **Unmounting:** `componentWillUnmount`.

Functional equivalents use `useEffect()` for mount/update/unmount handling.

6. Which language is used in React?

React uses **JavaScript** (ES6+), often with **JSX** syntax (HTML-like syntax embedded in JS).

7. How does React re-render?

When state or props change, React triggers a diff in the virtual DOM and re-renders only affected components.

8. How does the “Remember Me” function store data?

Usually stores tokens in:

- **Cookies** (with `HttpOnly` flag for security)
- **LocalStorage** or **SessionStorage**
The token keeps the user logged in even after page reload.

9. What is `preventDefault()` and why is it needed?

Stops the browser’s default event behavior.

Example: prevent form from refreshing on submit.

```
js
form.addEventListener('submit', e => {
  e.preventDefault();
});
```

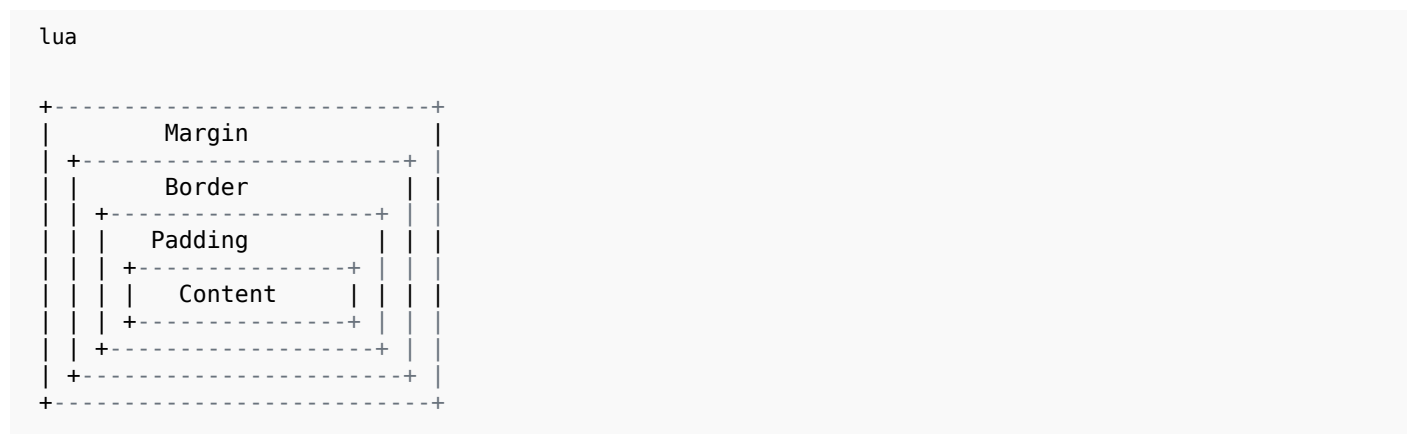
10. React JS vs Vue JS

Feature	React	Vue
Type	Library	Framework
DOM	Virtual DOM	Virtual DOM
Learning Curve	Steeper	Easier
State Management	Redux/Zustand	Vuex/Pinia

Feature	React	Vue
Popularity	Higher	Moderate

11. Explain and draw the box model of CSS

The **CSS Box Model** consists of:



It defines how space and layout are computed around elements.

[IMPLEMENTATION + WEB PERFORMANCE QUESTIONS]

1. How can you validate your email?

Use regex in frontend or backend:

```
js
```

```

/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email)

```

But always validate on **server-side** for security.

2. Suppose the website is fast and you want to make it slow.

- Add `setTimeout` or `sleep()` in API calls.
- Insert large blocking JS loops.
- Load large unused assets or huge images.
- Disable caching.

3. How to make a website faster?

- Enable caching (HTTP headers, CDN).
- Compress assets (gzip, brotli).
- Use lazy loading.

- Minify CSS/JS.
- Optimize images.
- Use efficient DB indexing and API pagination.

4. Best method to verify email: client or server-based?

Server-based.

Client-side can be bypassed. Server-side confirms the email actually exists via verification link or SMTP ping.

5. Name some servers you've used.

- **Web Servers:** Nginx, Apache, Express.js
- **App Servers:** Node.js, Spring Boot
- **Cloud:** AWS EC2, Azure App Service

6. How to make a YouTube-like automatic video quality controller?

- Monitor **network speed** using the **Network Information API** or custom bandwidth check.
- Maintain multiple resolutions (240p, 480p, 720p).
- Switch quality dynamically using **MediaSource API** when latency drops.

Pseudo:

```
js
if (bandwidth < 2Mbps) switchTo("480p");
```

7. How to design Captcha from scratch (no DB)?

- Generate random string (A-Z, 0-9).
- Render as image (Canvas API).
- Store value in **session** (server memory).
- On form submit, compare user input with session-stored code.

8. Handling high traffic causing lag

- Use load balancers (NGINX, AWS ELB).
- Enable caching (Redis).
- Use CDN for static content.
- Optimize DB with indexing and read replicas.

- Scale horizontally (multiple servers).

9. Write image slider with vanilla JavaScript

```
js

let i = 0;
const imgs = document.querySelectorAll('.slide');
setInterval(() => {
  imgs[i].classList.remove('active');
  i = (i + 1) % imgs.length;
  imgs[i].classList.add('active');
}, 3000);
```

10. Fetch millions of tweets of a public user

Use Twitter API with pagination and rate limit handling.

For full crawl: use **streaming API** + **database queue** + **batch storage** in cloud (e.g., S3 or MongoDB).

11. Implement traversing in video

Track current timestamp and move forward/backward using `HTMLVideoElement.currentTime`.

```
js

video.currentTime += 10; // skip 10 seconds
```

12. Throttling a website intentionally

Use artificial `setTimeout` or slow down server responses using middleware:

```
js

app.use((req, res, next)=>setTimeout(next, 3000));
```

13. Troubleshooting a slow website despite good hardware

Likely causes:

- Blocking synchronous scripts
- Poor DB query optimization (no indexes)
- Memory leaks or poor caching strategy
- High network latency from missing CDN
- Large image/video payloads

Use **Lighthouse** or **Chrome DevTools Performance** to diagnose.

14. OTP Verification without backend

Use a front-end random code + localStorage simulation:

```
js
const otp = Math.floor(100000 + Math.random() * 900000);
alert(`Your OTP: ${otp}`);
```

Then ask the user to enter it back within one session.

Not secure, only for mock demos.

15. Lift Algorithm (Elevator)

Use **priority queues**:

- Upward → min-heap
 - Downward → max-heap
- Each request added based on floor number and direction.
Lift serves in same direction until empty, then reverses.
-

[COMPUTER NETWORKS & SECURITY]

1. What is Encryption and Decryption?

Encryption: plaintext → ciphertext.

Decryption: ciphertext → plaintext (using a key).

2. What if encryption and hashing are disabled?

All data would be stored and transmitted in plaintext.

Leads to **total compromise** of credentials, privacy, and integrity.

3. What is CDN?

A **Content Delivery Network** caches and delivers content (images, videos, static files) from servers close to the user.

How it works:

- User request → directed to nearest CDN edge node.
- If cache miss → fetched from origin and stored.

Benefits: Low latency, load reduction, and faster delivery.

4. Can devices connected to the same router have the same IP?

Yes — same **public IP**, different **private IPs** (via NAT).

5. Can two computers have the same public IP?

Yes, if they share a common router or NAT gateway.

6. How router assigns IP to computers?

Using **DHCP protocol** — assigns dynamic private IPs from a local pool (e.g., 192.168.x.x).

7. What happens when you hit a URL while connected to a router?

1. Browser resolves domain using DNS.
2. Router forwards request via NAT.
3. Server responds → router maps it back to your private IP.
4. Browser renders HTML response.

8. What if two computers connected to the same router are assigned the same IP?

IP conflict → one or both lose network connectivity.

DHCP usually prevents this by managing IP leases.

9. Algorithms for encryption and hashing

Type	Algorithms
Symmetric	AES, DES, Blowfish
Asymmetric	RSA, ECC
Hashing	SHA-256, bcrypt, MD5 (obsolete)

10. Do all hashing algorithms produce fixed-size output?

Yes.

E.g., SHA-256 → 256-bit, SHA-512 → 512-bit, MD5 → 128-bit.

11. Symmetric vs Asymmetric Encryption

Type	Key Use	Example
Symmetric	Same key for encrypt/decrypt	AES
Asymmetric	Public-private key pair	RSA

12. Purpose of salting

To make hashes unique even for identical passwords, defending against rainbow tables.

13. Private and Public Keys

- **Private Key:** Secret, used to decrypt or sign.
- **Public Key:** Shared, used to encrypt or verify.

14. For storing passwords: encryption or hashing?

Use **hashing + salting** (e.g., bcrypt).

Encryption allows reversal — insecure for passwords.

15. Client-side vs Server-side validation

Type	Description	Risk
Client-side	Runs in browser using JS	Can be bypassed
Server-side	Validated on backend	Secure & required

16. Basics of server-side vs client-side scripting

Type	Language	Runs	Example
Client-side	JS	Browser	Form validation
Server-side	Python, Node, Java	Server	DB queries, authentication

17. Man-in-the-Middle Attack

An attacker intercepts communication between two parties.

Mitigation: use **HTTPS (TLS)**, **HSTS**, and **certificate pinning**.

18. Working of Browser

1. Parse HTML → build DOM.
2. Load CSS → build CSSOM.
3. Combine DOM + CSSOM → Render Tree.
4. Layout → Paint → Composite → Display.

19. Mail services and protocols (OSI layer info)

Service	Protocol	OSI Layer
Email sending	SMTP	Application
Email receiving	POP3, IMAP	Application

20. Horizontal vs Vertical Scaling

Type	Definition	Example
Horizontal	Add more servers	Load balancing with multiple instances
Vertical	Add more resources (CPU/RAM) to one server	Upgrading EC2 instance type

21. HTTP vs HTTPS

Protocol	Security	Port	Encryption
HTTP	None	80	No
HTTPS	TLS/SSL	443	Yes

22. WhatsApp uses encryption or hashing and why?

Encryption.

End-to-end encryption (Signal protocol) ensures only sender and receiver can read messages. Hashing wouldn't allow retrieval of the original message.

✓ Final Summary

You now have a **complete technical prep document** covering:

- Git & GitHub
- Hashing, Encryption, Cryptography
- DBMS Indexing
- React & Browser fundamentals
- Web performance & design logic
- CN & Security

Would you like me to format all of this as a **searchable PDF cheat sheet** (topic-wise sections, bullet summaries, ready for rapid prep)?



Answer all of these:-----

What are IIFEs in JavaScript?

What is the difference between React and AJAX ?

1.What is react? 2.Is react library or framework? 3.Advantages of React 4. What is Virtual Dom 5 What is Shadow Dom 6.What are stateful and stateless components in react? 7. Rules of React hooks 8. If component is still unmounted how will you change it 9. How can you cache in React?

10. What is Git and Github? 11. Can we use Git without Github 12. What is git blame command 13. What is git Cherry-pick command 14. If i done some changes in my files, but the file is in wrong branch and i have not committed it , how can i move the changes to the correct branch. 15. difference between HEAD, working tree and index, in Git? 16. How can i make changes in a file after committing 17. If i have some files which i dont have want share,how can i do it in git 18.What is function/difference of each command (add, commit and push)? 19. Can i write plain Javascript in React js 20. If i have to store my password in database securely, what should i do 21.what is hashing 22. If my password is "123" and your password is "12" will they have same hash length 23. What is the lenght of Sha 256 hashed password 24. If my password if "hello"and your is also "hello" will both be same. 25. How to uniquely identify same hashed password 26. What is salting 27. What is encrption and types of encrption 28. How does whatsapp uses asymmetric encrption 29. What is session and cookie 30. What if user disables cookie, what will happen 31. Can we store cookie for lifetime 32. What is default lifetime of session and cookie 33. Explain the process of captcha 34. Why to use session to store captcha 35. How to do client side and server side validation 36. How to make sure no malicious data is added to our database 37. What is Sanitization and why it is needed 38. What is Cross Site Scripting and give a real life example of it 39. How does youtube seamlessly streams video ,explain the backend. 40. Interviewer share his screen and shown image of 3 different elevator and asked to design the algorithm for it and which data structure can be used. 41. How to increase the speed of a website 42. Can two device have same Ip address 43. How do Isp providers differentiate which ip should be given 44. If you have a blog website whoes users are growing daily , how will you handle it. 45. How to make a website work slow 46. How to debug PHP code 47. How to do Unit testing in PHP 48.what is indexing 49. What are it drawbacks 50. Can we index multiple columns 51. What is join and it types 52 . Explain Left and Inner Join 53. What are pseudo elements in css 54. What are different type of displaying methods in css and explain them 55. What are selectors in css 56. What is a CSS preprocessor? 57. what is chromium 58. What you will do to make your css compatible to all the browsers 59. What are react mixins 60. What are uncontrollable components in react 61. If we use plain js in react will it directly manipulate DOM 62. If i have a blog website what will it store in the CDN

How captcha works?

How can you validate your email? Hashing and encryption, symmetric and asymmetric, algorithms use, which is faster react or js.

Good. Straight answers, no fluff. I'll hit each question directly and practically so you can use this for interviews.

1. What are IIFEs in JavaScript?

An IIFE is an Immediately Invoked Function Expression. It's a function defined and executed right away to create a private scope and avoid polluting globals. Example:

```
(function(){ /* code */ })();
```

2. What is the difference between React and AJAX?

React is a UI library for building component-based user interfaces. AJAX is a technique for making asynchronous HTTP requests from the browser. React manages the view; AJAX handles data fetching. They are complementary.

React block (short, numbered as you asked)

1. What is React?

A JavaScript library for building user interfaces using components and a virtual DOM.

2. Is React a library or framework?

Library.

3. Advantages of React

Component reusability, virtual DOM performance, large ecosystem, good for SPAs, strong community and tooling.

4. What is Virtual DOM?

An in-memory representation of the real DOM. React diffs virtual DOM snapshots and applies minimal updates to the real DOM.

5. What is Shadow DOM?

A browser feature used by web components to provide encapsulated DOM and styles separate from the main document.

6. Stateful vs Stateless components

Stateful components hold local state (class with state or functional with hooks). Stateless components receive props and do not manage internal state.

7. Rules of React hooks

Call hooks only at top level of React functions and only inside React function components or custom hooks. Do not call inside loops, conditions, or nested functions.

8. If component is still unmounted how will you change it

You cannot modify an unmounted component. If you need to set state after async work, check mounted status or cancel/cleanup in `useEffect` cleanup before calling `setState`.

9. How can you cache in React?

Use memoization (`React.memo`, `useMemo`), local state, Context, IndexedDB, localStorage, service workers, or client-side caches like SWR/React Query.

Git / GitHub / Git workflows

10. What is Git and GitHub?

Git is a distributed version control system. GitHub is a hosting service for Git repositories with collaboration features.

11. Can we use Git without GitHub?

Yes. Git runs locally. Remotes can be any server or none at all.

12. What is `git blame`?

Shows who last modified each line of a file with commit hashes and timestamps.

13. What is `git cherry-pick`?

Apply a single commit from another branch onto the current branch by creating a new commit with the same changes.

14. If I changed files on wrong branch but not committed, how to move changes to correct branch?

Stash or switch with index preservation:

- `git stash push -m "wip"` then `git checkout correct-branch` then `git stash pop`
Or: `git checkout correct-branch -- <file>` if you want per-file moves.

15. Difference between HEAD, working tree and index (staging area)

HEAD: pointer to current commit. Working tree: files in your filesystem. Index: staged snapshot that will be committed next.

16. How can I make changes in a file after committing?

Make edits, then `git add` and `git commit` to create a new commit. To alter last commit: `git commit --amend` (rewrites history locally).

17. If I have files I do not want to share, how to ignore them in Git?

Add patterns to `.gitignore`. If already tracked, remove from index: `git rm --cached <file>` then commit.

18. Function/difference of add, commit, push

`git add` moves changes to index. `git commit` records staged changes into local history. `git push` uploads local commits to remote.

19. Can I write plain JavaScript in React?

Yes. React components are JS. You can use plain JS, DOM APIs, or other libs, but avoid direct DOM manipulation that conflicts with React rendering.

Passwords, hashing, encryption, salting

20. If I have to store password securely in DB what should I do?

Hash with a slow, salted algorithm like bcrypt/Argon2/scrypt with appropriate cost parameters. Never store plaintext. Don't use general-purpose fast hashes like MD5 or SHA-1 alone.

21. What is hashing?

One-way function that maps input data to fixed-length output. Used for integrity and password verification.

22. If my password is "123" and yours is "12" will they have same hash length?

Yes. Hash outputs have fixed length regardless of input length.

23. What is the length of SHA-256 hashed password?

SHA-256 produces 256 bits. Represented in hex it is 64 characters.

24. If my password is "hello" and yours is also "hello" will both be same?

Yes, same input and same salt produce same hash. If salted with unique salts, hashes differ.

25. How to uniquely identify same hashed password

Use a per-user salt stored alongside the hash. You identify by comparing `hash(password + salt)` for that user. If salt differs, same plain password produces different hashes.

26. What is salting?

Adding a random value to a password before hashing to make precomputation attacks ineffective and ensure identical passwords produce different hashes.

27. What is encryption and types of encryption?

Encryption is reversible transformation using keys. Types: symmetric (single shared secret, e.g., AES)

and asymmetric (public/private keys, e.g., RSA, ECC). Often combined: asymmetric to exchange symmetric keys, symmetric for bulk data.

28. **How does WhatsApp use asymmetric encryption?**

WhatsApp uses the Signal Protocol. Asymmetric keys are used for identity and to establish session keys. After key agreement, messages are encrypted with symmetric session keys (double ratchet). Asymmetric is primarily for key exchange and authentication, not for each message payload.

Sessions, cookies, captcha, validation, security

29. **What is session and cookie?**

Cookie: client-side small data stored in browser. Session: server-side data associated with a session id, often stored in server memory or DB. Cookie often stores the session id.

30. **What if user disables cookie, what will happen?**

Session via cookie breaks. Alternatives: URL tokens, Web Storage with manual transmission, or OAuth flows, but disabling cookies disables many auth/session patterns and reduces security.

31. **Can we store cookie for lifetime?**

You can set a long expiration. Not recommended for sensitive data. Use secure, HttpOnly, SameSite, and renew/rotate tokens.

32. **Default lifetime of session and cookie**

No single default. Browser session cookies last until browser closes. Server sessions depend on server config (commonly 20–30 minutes inactivity). Cookies with **Expires** persist until that date.

33. **Explain process of captcha**

Server generates random token, renders challenge (image/audio), stores expected answer in server-side session or ephemeral store, client submits response, server validates answer and expires token. Use unpredictability and rate limits.

34. **Why use session to store captcha?**

To keep the expected answer secure on server side and tie it to that user session to prevent replay and tampering.

35. **How to do client-side and server-side validation**

Client-side: quick UX checks with JS. Server-side: authoritative checks on every request. Always implement server-side validation even if you have client checks.

36. **How to make sure no malicious data is added to DB**

Validate input, sanitize or parameterize queries, use prepared statements, apply least privilege DB accounts, enforce type and length checks, use WAF and rate limiting.

37. **What is sanitization and why needed**

Removing or encoding malicious parts of input to prevent injection attacks. Example: HTML-encode output to prevent XSS.

38. **What is Cross Site Scripting and real life example**

XSS is injecting malicious scripts into pages served to other users. Example: an attacker posts `<script>stealCookie()</script>` in a forum post; when others view it, the script runs and steals their session cookies.

Video streaming, CDN, backend

39. **How does YouTube seamlessly stream video, explain the backend**

Store multiple encoded resolutions/chunks (HLS/DASH). Use a CDN to cache chunks worldwide. Player detects bandwidth and switches streams (ABR). Backend handles ingestion, transcoding, chunk storage, and metadata services. Use load balancers, origin storage (object store), authentication and telemetry.

Elevator/elevator algorithm and data structure

40. **Design algorithm for elevators and which data structure**

Use two priority queues per elevator: up as min-heap of floors greater than current, down as max-heap of floors less than current. Serve requests in current direction until queue empty, then switch direction. Use boolean arrays or bitsets for quick request checks. Add scheduling for multiple elevators: nearest-car-first or load-balancing with cost function.

Website speed, IPs, scaling

41. **How to increase speed of a website**

Use CDN, caching, optimize assets, compress, lazy load, minimize JS, fix long tasks, optimize DB queries and indexing, use HTTP/2 or HTTP/3, use service workers.

42. **Can two devices have same IP address?**

They can share the same public IP behind NAT but not the same private IP on the same network without conflict.

43. **How do ISPs differentiate which IP to give?**

ISPs use DHCP pools and assign IPs based on available addresses and MACs. For public IPs, they allocate from their assigned ranges and use routing to differentiate.

44. **If a blog's users grow daily how to handle it**

Horizontal scale web servers, use CDN for static assets, cache DB reads, add read replicas, use message queues for background jobs, shard or partition DB if needed, use autoscaling.

45. **How to make a website work slow**

Introduce artificial delays server-side, block main thread with heavy JS, disable caching, load large resources synchronously.

PHP, testing, indexing, SQL joins

46. **How to debug PHP code**

Use error reporting, `xdebug`, step-through debugging, logs, `var_dump/print_r` for quick checks, unit tests.

47. **How to do unit testing in PHP**

Use PHPUnit, write tests for functions/classes, mock external dependencies, run in CI.

48. **What is indexing**

DB index is a data structure (usually B-tree) that speeds up data retrieval by providing fast lookup.

49. **What are its drawbacks**

Indexes add write overhead, consume disk space, can be misused causing poor performance, and

increase complexity for inserts/updates.

50. **Can we index multiple columns?**

Yes. Composite indexes support searches on multiple columns. Use order-aware indexing; leftmost prefix matching matters.

51. **What is join and its types**

Join combines rows from two tables based on related columns. Types: INNER, LEFT (LEFT OUTER), RIGHT (RIGHT OUTER), FULL OUTER, CROSS JOIN, SELF JOIN.

52. **Explain Left and Inner Join**

Inner join returns rows where keys match in both tables. Left join returns all rows from left table and matched rows from right, **NULL** where no match.

CSS and compatibility

53. **What are pseudo elements in CSS**

Selectors that style parts of elements like `::before`, `::after`, `::first-letter`, `::first-line`.

54. **Different display methods in CSS and explain**

`display: block` (new line, takes full width), `inline` (flows within line), `inline-block` (inline but width/height apply), `flex` (flexbox layout), `grid` (grid layout), `none` (hidden).

55. **What are selectors in CSS**

Patterns to select elements: element, class (`.a`), id (`#id`), attribute, pseudo-classes, combinators.

56. **What is a CSS preprocessor?**

Tools like SASS, LESS that add variables, nesting, mixins and compile to CSS.

57. **What is Chromium?**

Open source browser project that powers Chrome and other browsers. Provides rendering engine and browser features.

58. **What you will do to make your CSS compatible to all browsers**

Use vendor prefixes where needed, test in target browsers, use feature detection (not user agent), transpile CSS via PostCSS/Autoprefixer, avoid bleeding-edge features or provide fallbacks.

59. **What are React mixins**

Old pattern for sharing code in early React class components. Deprecated in favor of higher-order components and hooks.

60. **What are uncontrolled components in React**

Components where form data is handled by the DOM, not React state. Use refs to access values.

61. **If we use plain JS in React will it directly manipulate DOM?**

Yes, direct DOM APIs alter the real DOM. Do that cautiously because it can conflict with React's virtual DOM. Use refs for controlled integration.

62. **If I have a blog website what will it store in the CDN**

Static assets: images, CSS, JS bundles, fonts, video segments, and sometimes pre-rendered HTML or API edge cache responses.

Captcha, email validation, hashing vs encryption, speed

How captcha works?

Server generates challenge and expected response, stores answer server-side tied to session or ephemeral token, sends challenge to client, client submits response, server validates and expires token.

How can you validate your email?

Frontend: basic regex for format. Server: verify format then send email with unique token link. Confirm only when user clicks link. Optionally use MX checks or SMTP probe but send-confirm is standard.

Hashing and encryption

Hashing: one-way, fixed-length output, used for integrity and password verification. Encryption: reversible with keys, used for confidentiality.

Symmetric and asymmetric

Symmetric uses same key for encrypt/decrypt, fast, good for bulk data. Asymmetric uses public/private keys, enables secure key exchange and signatures, slower.

Algorithms used

Symmetric: AES, ChaCha20. Asymmetric: RSA, ECC. Hashing: SHA-2 family, SHA-3, bcrypt, Argon2 for password hashing.

Which is faster: React or JS?

Bad question. React is a JS library. Pure vanilla JS DOM manipulation can be faster for trivial tasks. React adds abstraction and diffing overhead but gives huge development and maintenance benefits. For raw microbenchmarks vanilla JS can be faster; in real apps React often yields better productivity and acceptable performance.

If you want, I can:

- Convert this into a printable one-page cheat sheet.
- Expand any single answer with code examples, diagrams, or sample interview follow-ups.

Which do you want next?